Neural Rover: Design and Implementation of a Wi-Fi-Controlled Mobile Robot with Live Pan–Tilt Video

Jubayed Ahmed Sahel, Fahad Hossain Shourov,
Pujon Chakraborty, and Anonna Rahman Spondon
Department of Electrical and Computer Engineering
North South University, Dhaka, Bangladesh
Supervisor: Dr. K. M. A. Salam

***Abstract*—This paper presents the completed CSE499A implementation of *Neural Rover*, a four-wheel mobile robot controlled from a browser over a 2.4-GHz Wi-Fi network. A 30-pin ESP32 serves a responsive WebSocket controller, drives four DC geared motors through two L298N dual H-bridges, commands a two-axis camera mount, and applies fail-safe stopping. A separate ESP32-CAM captures the scene and supplies a live stream embedded in the same control interface. The battery feeds the motor drivers directly while an LM2596 module produces a regulated 5-V logic rail. The final build was validated through movement, steering, continuous-command, stop, video, pan–tilt, discovery, and reconnection tests. All hardware construction, firmware integration, networking, safety behavior, testing, documentation, and demonstration tasks specified for CSE499A are complete.**

***Index Terms*—ESP32, ESP32-CAM, mobile robot, WebSocket, L298N, pan–tilt camera, remote control.**

I. Introduction

Mobile robots used for inspection, teaching, and response experiments require both controllable motion and dependable visual feedback. Wireless reliability is especially important because degraded communication can directly affect an operator's ability to command and observe a remote platform; NIST has therefore developed repeatable methods for measuring response-robot performance under attenuated radio conditions [1]. Neural Rover addresses this control-and-observation problem with a compact, locally hosted system that does not require an Internet cloud service.

The ESP32 was selected as the primary controller because a single device combines 2.4-GHz Wi-Fi, programmable GPIO, multiple UART interfaces, and hardware PWM peripherals [2]. A second ESP32 camera board separates image capture and streaming from time-critical motor control. Espressif's camera driver supports common ESP32 camera sensors, frame-buffer management, and conversion paths needed for browser streaming [3]. This two-board organization keeps vehicle control available even while the camera is encoding frames.

The purpose of the CSE499A project was to design, build, and verify a complete rover platform that can be driven from a phone or laptop, show live video on the controller page, aim the camera using pan and tilt, and enter a safe stopped state if control messages disappear. The project-specific firmware and implementation record are maintained by J. A. Sahel in the public CSE499 repository; this report corresponds to reviewed commit `201e9b3` [7].

The eight-week workplan covered component selection, chassis assembly, motor wiring, browser control, camera streaming, mechanical pan–tilt integration, communication reliability, full-system testing, demonstration, and reporting. Every task required for CSE499A has been implemented and checked, giving 100% completion of the planned CSE499A scope [7]. The present paper documents that completed system; future computer-vision extensions are outside this report.

II. Purpose and Design Objectives

The central objective was to provide intuitive teleoperation and useful visual awareness with inexpensive, widely available modules. The rover was therefore required to: (1) move forward and backward and turn left and right; (2) react immediately to a held browser control; (3) stop on release, timeout, browser loss, or Wi-Fi loss; (4) display the camera stream on the same page; (5) aim the view with two servos; and (6) expose understandable diagnostic endpoints. WebSocket communication was selected for motion commands because it provides a persistent two-way connection over TCP, avoiding a new HTTP transaction for every repeated command [6].

The resulting platform is suitable for indoor visual inspection, laboratory teleoperation, mobile-robot networking experiments, and embedded-systems education. These applications use the same completed capabilities—remote motion, camera feedback, two-axis viewing, local networking, and fail-safe stopping—without changing the base hardware. Such uses are consistent with the broader need to evaluate response robots as integrated mobility, sensing, and communication systems [1].

III. Methodology

A. System Architecture

Fig. 1 separates the system into control, video, actuation, and power paths. The phone or laptop and both ESP32 boards join the same 2.4-GHz `CAR` network. The browser opens the main ESP32 control page on port 80 and maintains `/ws` for

TABLE I
FINAL MAIN-ESP32 PIN ASSIGNMENT

Subsystem	Signal	GPIO
Left L298N	ENA, IN1, IN2	32, 23, 22
	IN3, IN4, ENB	21, 19, 18
Right L298N	ENA, IN1, IN2	25, 26, 27
	IN3, IN4, ENB	14, 12, 33
Pan–tilt	Pan, tilt	13, 4
UART2	RX2, TX2	16, 17

control messages. The ESP32-CAM serves its camera page on port 80 and its MJPEG stream on port 81. UART2 lets the camera board report its current IP address to the main controller, which then supplies the correct stream address to the browser [7].

B. Motor and Servo Interfaces

Two L298N modules provide four H-bridge channels. The L298 is a dual full-bridge driver with independent enable inputs, making each module appropriate for two bidirectional DC motors [4]. The enable jumpers were removed so the ESP32 can drive ENA and ENB with 12-kHz, 8-bit PWM. Table I gives the final physical wiring. It supersedes early left/right labels: the GPIO32 group is the physical left driver, and the GPIO25 group is the physical right driver. The front-left polarity is corrected in software, so all wheels now agree with the Forward command [7].

The left driver OUT1/OUT2 pair feeds the front-left motor and OUT3/OUT4 feeds the rear-left motor. The right driver uses the same channel order for the front-right and rear-right motors. Pan and tilt signals originate from GPIO13 and GPIO4 of the *main* ESP32, while the servos receive 5 V from the regulated rail. UART2 is cross-connected: main RX2 (GPIO16) receives ESP32-CAM U0T, and main TX2 (GPIO17) drives ESP32-CAM U0R [7].

C. Power Distribution and Electrical Safety

The 3S battery supplies both L298N motor inputs and the LM2596 input. The LM2596 is a 150-kHz step-down regulator family rated for loads up to 3 A when correctly implemented and thermally managed [5]. Its output is adjusted to 5.0 V before connection and powers the ESP32 logic, ESP32-CAM, and servos. Battery negative, regulator negative, both controllers, both motor drivers, and both servo grounds are joined to form the signal reference [7].

Motor current transients previously produced resets and corrupted serial output, so the final procedure separates motor power from regulated logic power and avoids competing USB and buck-converter 5-V sources. ENA and ENB PWM jumpers remain removed; the L298N 5V-EN jumper is treated separately according to the chosen logic-supply configuration. The rover is tested with a charged battery, short power conductors, secure grounds, and wheels lifted for first motion after wiring changes [4], [5], [7].

D. Firmware, Network, and Safety Logic

The main ESP32 firmware joins `CAR`, starts an asynchronous web server, and exposes a persistent WebSocket at `/ws`. The browser sends `F`, `B`, `L`, or `R` while a control is held, `S` on release, and periodic `K` keepalive messages. The protocol follows the full-duplex WebSocket model defined in RFC 6455 [6]. The page also sends `P0-P180` and `T0-T180` servo commands and requests the camera address from `/camera-ip` [7].

Motor output begins at duty 120 and ramps by 12 counts every 12 ms toward 255. A direction reversal first removes PWM, waits 40 ms, changes the H-bridge inputs, and then ramps again. If no valid movement or keepalive message arrives within 1300 ms, or if the browser or Wi-Fi disconnects, all enable outputs are set to zero. These choices provide usable starting torque while preventing stale commands from keeping the rover moving [7].

The camera firmware first attempts hardware-JPEG capture. The installed sensor reports that hardware JPEG is unavailable, so the code falls back to RGB565 at QVGA (320×240) and converts frames to JPEG in software for streaming. Espressif notes that RGB/YUV modes place greater load on the chip and that lower resolutions are preferable without JPEG support, which supports the stability-oriented QVGA choice [3]. Fixed names `rover.local` and `rovercam.local` are offered through mDNS, while numeric DHCP addresses printed to serial provide a fallback [7].

E. CSE499A Workplan Completion

Table II maps the original eight-week workplan to the verified CSE499A result. Where early planning used a generic communication or alert description, the implemented deliverable is the local WebSocket, UART discovery, and fail-safe control layer that was integrated and tested on the physical rover. All listed CSE499A tasks are complete [7].

IV. RESULTS, DISCUSSION, AND ANALYSIS

Functional testing used the final physical wiring and firmware summarized above. Each check in Table III was repeated after correcting the physical driver labels and front-motor direction. The outcome is a fully controllable rover in which forward/backward translation, turning, released-command stopping, camera display, pan–tilt positioning, address discovery, and connection recovery operate as intended [7].

A. Motion-Control Analysis

The most important motor result was not a change in the physical wiring but an accurate mapping between physical sides, channels, and software names. Early diagrams labeled the two driver groups oppositely, and the front-left motor required reversed logical polarity. Recording the final mapping and applying the correction in one software layer removed ambiguity: Forward now produces forward rotation on all four wheels, while the rear motors retain their already correct polarity [7].

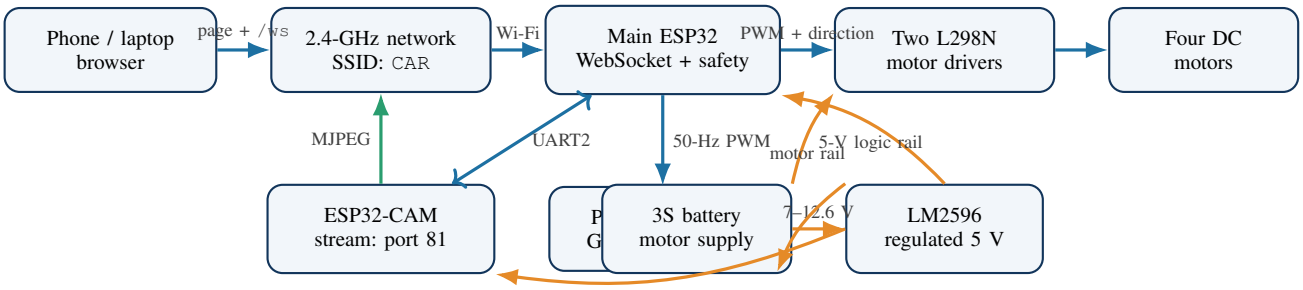


Fig. 1. Implemented Neural Rover architecture and separation of control, video, actuation, and power paths.

TABLE II
CSE499A WORKPLAN AND COMPLETION STATUS

Week	Planned focus	Completed evidence	Status
1	Components and circuit plan	Components selected; power, pin, and system diagrams documented	Complete
2	Chassis and motor wiring	Four motors mounted; two L298N modules wired and individually tested	Complete
3	Phone control	Browser motion controls, WebSocket commands, steering, and stop implemented	Complete
4	Camera streaming	ESP32-CAM initialized; camera page and port-81 stream integrated	Complete
5	Pan–tilt mount	Two-axis mount installed and controlled from the main webpage	Complete
6	Communication and safety	UART camera discovery, keepalive, timeout, disconnect stop, and diagnostics implemented	Complete
7	Integration testing	Motion, video, servos, network recovery, and power behavior tested together	Complete
8	Final demonstration/report	Working system, final firmware, presentation, pin document, and IEEE report prepared	Complete

TABLE III
INTEGRATED FUNCTIONAL TEST RESULTS

Test	Acceptance observation	Result
Forward/backward	Four wheels agree with command	Pass
Left/right turn	Correct differential direction	Pass
Held command	Motion continues while held	Pass
Release/timeout	Outputs return to stopped state	Pass
Live video	Stream visible in control page	Pass
Camera discovery	UART address reaches main page	Pass
Pan and tilt	Both axes respond to controls	Pass
Reconnect	Control resumes after reconnection	Pass

The persistent WebSocket also explains the improved driving behavior. Press-and-hold events can update the controller over an existing full-duplex channel, and the browser does not navigate to a separate page for each command [6]. The ramp limits abrupt current demand, the 40-ms dead time protects direction reversal, and the 1300-ms watchdog ensures that a lost browser cannot leave the last motion latched. Together these mechanisms favor predictable control over maximum instantaneous acceleration [7].

B. Video and Network Analysis

Separating camera and motor workloads proved useful. The main ESP32 handles movement and safety independently, whereas the ESP32-CAM spends its resources on capture and software JPEG conversion. The QVGA fallback reduces processing and memory pressure, but its frame rate and detail are necessarily below a compatible hardware-JPEG camera operating at a higher resolution. This is an explicit engineering tradeoff for the installed sensor rather than a browser zoom setting [3], [7].

DHCP addresses can change when the hotspot reconnects. The implementation therefore provides both mDNS names and serially reported numeric endpoints, while UART informs the main controller of the current camera address. The browser may briefly log repeated connect/disconnect events during refreshes or network changes; the safety logic converts these events into a motor stop rather than uncontrolled motion. Reliable radio conditions remain a system-level concern for any teleoperated robot [1], [7].

C. Power and Integration Analysis

The investigation of intermittent resets showed that successful bench commands alone do not prove correct vehicle-level power behavior. When motors start, their transient load and switching noise can disturb logic power or ground. Supplying the L298N motor rail from the battery, deriving a separate regulated 5-V rail through the LM2596, and maintaining a

common reference produced the correct functional arrangement. The L298N still introduces conduction loss typical of a bipolar H-bridge, so battery charge, wiring resistance, module temperature, and motor stall current remain practical operating limits [4], [5].

No artificial latency or frame-rate measurements are reported because the CSE499A acceptance work was functional rather than instrumented. The evidence supports the stated pass/fail results and completion status, while quantitative network latency, radio range, current consumption, and endurance remain appropriate subjects for a later controlled measurement campaign.

V. Conclusion

Neural Rover has reached the complete CSE499A target: a four-wheel vehicle can be driven from a browser, observed through an integrated live camera, and aimed using pan and tilt. The final firmware resolves the physical left/right and front-motor polarity issues, uses persistent commands for smooth control, and stops safely on release, timeout, browser loss, or network loss. The two-controller architecture keeps motor safety separate from camera processing, while the battery and regulated logic rails provide a clear power structure. All eight weeks of planned CSE499A hardware, firmware, integration, testing, demonstration, and reporting work are complete. The platform is ready for later extensions such as autonomous line following, object detection, mapping, or additional environmental sensors without changing the verified baseline described here.

References

[1] K. E. Kimble, R. Candell, and K. S. Saidi, "A test method for measuring response robot performance under attenuated radio conditions," NIST Interagency/Internal Rep. 8243, May 2019, doi: 10.6028/NIST.IR.8243.

[2] Espressif Systems, "ESP32 series datasheet," ver. 5.2, 2025. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.pdf

[3] Espressif Systems, "ESP32 camera driver," *esp32-camera*. [Online]. Available: https://github.com/espressif/esp32-camera

[4] STMicroelectronics, "L298 dual full-bridge driver," datasheet. [Online]. Available: https://www.st.com/resource/en/datasheet/l298.pdf

[5] Texas Instruments, "LM2596 SIMPLE SWITCHER power converter 150-kHz 3-A step-down voltage regulator," datasheet. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm2596.pdf

[6] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, Dec. 2011, doi: 10.17487/RFC6455.

[7] J. A. Sahel, "CSE499 Neural Rover project source code and documentation," GitHub repository, commit `201e9b3`, Aug. 6, 2026. [Online]. Available: https://github.com/Jubayed-Sahel/CSE499

APPENDIX A: FINAL IMPLEMENTATION REFERENCE

This appendix consolidates the verified build information used during assembly, upload, and demonstration. It introduces no new experimental claims and should be read with the system architecture, results, and cited sources in the main paper.

A. Component Functions

TABLE IV
PRINCIPAL COMPONENTS AND ROLES

Component	Implemented function
30-pin ESP32	Hosts the browser controller and WebSocket; drives motors and servos; receives the camera address.
ESP32-CAM	Captures RGB565 QVGA frames, converts them to JPEG, and serves the live stream.
Two L298N modules	Provide four reversible H-bridge motor channels.
Four geared motors	Produce independent front-left, rear-left, front-right, and rear-right wheel motion.
Two SG90 servos	Aim the camera in pan and tilt.
LM2596 module	Converts the battery rail to the regulated 5.0-V logic/servo rail.
3S battery	Supplies both motor drivers and the buck-converter input.

B. Physical Motor Wiring

TABLE V
DRIVER-TO-MOTOR INTERFACE

Driver	Channel	GPIO group	Motor
Left	ENA/IN1/IN2	32/23/22	Front-left
Left	ENB/IN3/IN4	18/21/19	Rear-left
Right	ENA/IN1/IN2	25/26/27	Front-right
Right	ENB/IN3/IN4	33/14/12	Rear-right

The final firmware reverses the front-left logical polarity while preserving the physical terminal wiring. ENA and ENB jumpers remain removed because the four enable pins receive ESP32 PWM. Pan uses main-ESP32 GPIO13, tilt uses main-ESP32 GPIO4, and both servos use the regulated 5-V rail with common ground.

C. Data and Network Interfaces

TABLE VI
FINAL LOCAL INTERFACES

Interface	Address or connection
Controller	http://rover.local/ or printed main-ESP32 IP
Motor channel	ws://rover.local/ws
Camera page	http://rovercam.local/
Camera stream	http://rovercam.local:81/stream
Camera discovery	Main RX2 GPIO16 ← CAM U0T
Camera command	Main TX2 GPIO17 → CAM U0R
Wi-Fi	SSID `CAR`, 2.4 GHz

Both boards receive dynamic IP addresses from the same network. The main controller prints its numeric page address, and the camera repeatedly announces `CAMIP:x.x.x.x` through UART2. The browser first requests `/camera-ip`; numeric endpoints remain available when a client cannot resolve the fixed `.local` names.

D. Firmware and Safety Parameters

TABLE VII
VERIFIED CONTROL PARAMETERS

Parameter	Final value
Motor PWM	12 kHz, 8 bit
Start/full duty	120 / 255
Ramp step/interval	12 / 12 ms
Direction dead time	40 ms
Command timeout	1300 ms
Servo PWM	50 Hz, 16 bit
UART2	115200 baud
Camera fallback	RGB565, 320×240

The stop state is applied when a control is released, a command becomes stale, the WebSocket disconnects, or Wi-Fi is lost. Direction reversal removes PWM before changing bridge polarity. This prevents a browser refresh or communication failure from leaving motion latched.

E. Upload and Demonstration Checklist

The main sketch is uploaded as *ESP32 Dev Module*; the camera sketch is uploaded as *AI Thinker ESP32-CAM*. GPIO0 is grounded only while placing the camera board in download mode and is disconnected before normal boot. Serial Monitor is set to 115200 baud.

Before demonstration, the LM2596 output is checked at 5.0 V, the battery is fully charged, all grounds are common, the wheels are initially lifted, and only one control page/stream client is opened. The acceptance sequence is: connect both boards, confirm both endpoints, test Stop, test each direction, hold Forward, test pan/tilt, view the stream, disconnect the browser, and verify automatic stopping.

F. CSE499A Handover State

The final handover consists of the working main-controller sketch, working camera sketch, tested pin map, power and upload instructions, browser-control protocol, presentation, and this five-page report. The documented configuration is the baseline for later line-following, mapping, object-detection, or sensor-expansion work.